EXPERIMENT 9

Write a Python program to implement the Travelling Salesman Problem

Register Number: **192511013**

192511013

Aim

To write a Python program that finds the shortest possible route that visits every city exactly once and returns to the starting city (Travelling Salesman Problem), using the brute-force method on a distance matrix.

Algorithm

Step 1: Store the cities and the distance matrix, where distance[i][j] is the distance from city i to city j.

Step 2: Select the starting city (city A).

Step 3: Generate all permutations of the remaining cities. Each permutation, with the start city added at the beginning and at the end, is one complete tour.

Step 4: For each tour, calculate its total cost by adding the distances between consecutive cities.

Step 5: Compare the cost with the best (minimum) cost found so far. If it is smaller, store this tour as the best tour.

Step 6: After all (n - 1)! permutations have been examined, the stored tour is the optimal tour.

Step 7: Print the distance matrix, the number of tours examined, the shortest tour and its minimum cost.

192511013

Program

```
from itertools import permutations

cities = ['A', 'B', 'C', 'D', 'E']

# distance[i][j] = distance from city i to city j
distance = [
    [0, 10, 15, 20, 25],
    [10, 0, 35, 25, 30],
    [15, 35, 0, 30, 20],
    [20, 25, 30, 0, 15],
    [25, 30, 20, 15, 0],
]

def tour_cost(tour):
    cost = 0
    for i in range(len(tour) - 1):
        cost += distance[tour[i]][tour[i + 1]]
    return cost

def tsp(start=0):
    n = len(cities)
    others = [i for i in range(n) if i != start]
    best_cost, best_tour = float('inf'), None
    count = 0
    for perm in permutations(others):
        tour = [start] + list(perm) + [start]
        cost = tour_cost(tour)
        count += 1
        if cost < best_cost:
            best_cost, best_tour = cost, tour
    return best_tour, best_cost, count

if __name__ == "__main__":
    print("Distance matrix:")
    print("     " + "  ".join(f"{c:>3}" for c in cities))
    for i, row in enumerate(distance):
        print(f"{cities[i]:>3}  " + "  ".join(f"{d:>3}" for d in row))
    tour, cost, count = tsp(start=0)
    print("\nTours examined:", count)
    print("Shortest tour :", ' -> '.join(cities[i] for i in tour))
    print("Minimum cost  :", cost)
```

192511013

Output

```
Distance matrix:
       A    B    C    D    E
  A    0   10   15   20   25
  B   10    0   35   25   30
  C   15   35    0   30   20
  D   20   25   30    0   15
  E   25   30   20   15    0

Tours examined: 24
Shortest tour : A -> B -> D -> E -> C -> A
Minimum cost  : 85
```

192511013